Part 3 – Cross-Language Design Pattern Analysis

Data-to-Science (D2S) Case Study

1 Application Description

Data-to-Science (D2S) is an open-source geospatial data management platform. Researchers upload, version, and share flight-mission datasets; experiments are processed through an automated workflow pipeline. Two cross-cutting design patterns underpin D2S: Strategy (for pluggable access-control policies) and Observer (for real-time workflow-status notifications).

This analysis reimplements those same two patterns against an equivalent toy domain (dataset access control and experiment workflow events) in four languages: Java, Python, JavaScript, and C++.

2 Framework Architecture

- **Java** – plain JDK 17; interface-based polymorphism.
- **Python** – stdlib + `abc.ABC`; duck-typing + protocols.
- **JavaScript** – Node.js 20; prototype chain, no explicit interfaces.
- **C++** – C++17; abstract base class with virtual/override.

3 Pattern Inventory

Pattern	Category	Intent	D2S Component
Strategy	Behavioural	Encapsulate interchangeable algorithms	`policy_engine.py`
Observer	Behavioural	Notify dependents on state change	WorkflowEngine / WebSocket

4 Pattern-to-Source-Code Mapping

Language	Strategy file	Observer file
Java	`code/java/Strategy.java`	`code/java/Observer.java`
Python	`code/python/strategy.py`	`code/python/observer.py`
JavaScript	`code/js/strategy.js`	`code/js/observer.js`
C++	`code/cpp/strategy.cpp`	`code/cpp/observer.cpp`

5 UML Diagrams (textual)

5.1 Class Diagram – Strategy Pattern

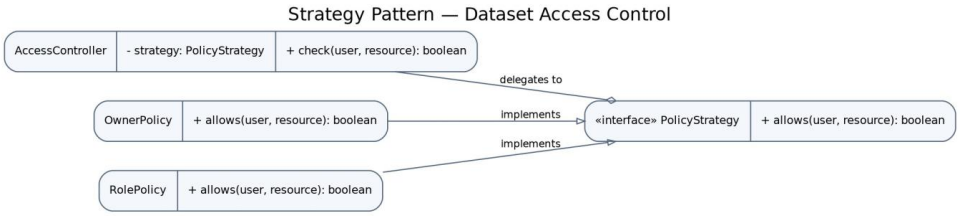


Figure 1: Strategy Pattern — Dataset Access Control

5.2 Class Diagram – Observer Pattern

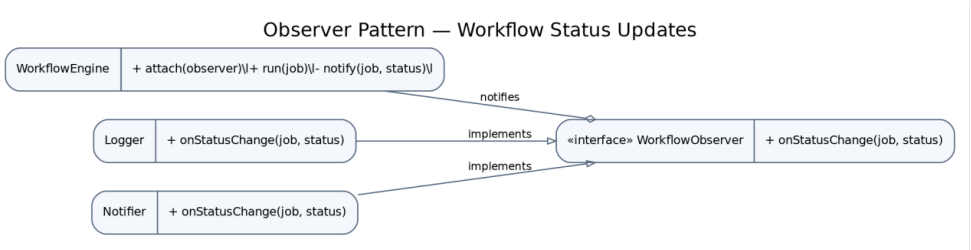


Figure 2: Observer Pattern — Workflow Status Updates

5.3 Sequence Diagram – Observer run()

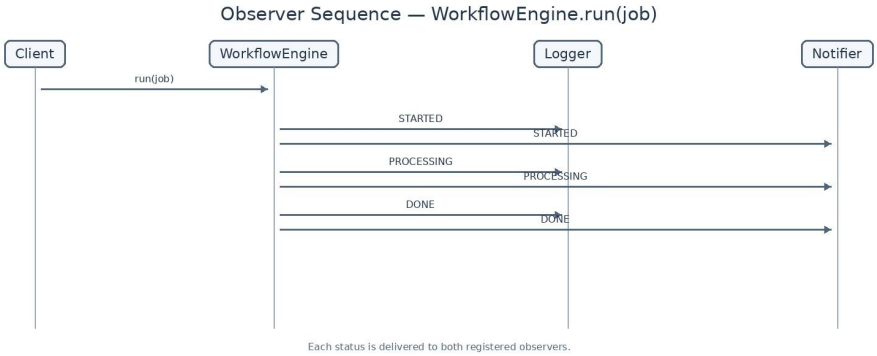


Figure 3: Observer Sequence — WorkflowEngine.run(job)

```
Client -> WorkflowEngine : run("job")
WorkflowEngine -> Logger : onStatusChange("job","STARTED")
WorkflowEngine -> Notifier : onStatusChange("job","STARTED")
WorkflowEngine -> Logger : onStatusChange("job","PROCESSING")
WorkflowEngine -> Notifier : onStatusChange("job","PROCESSING")
WorkflowEngine -> Logger : onStatusChange("job","DONE")
WorkflowEngine -> Notifier : onStatusChange("job","DONE") [prints notify]
```

6 Source-Code Evidence

6.1 Java – Strategy

Listing 1: Strategy.java (key excerpt)

```
interface PolicyStrategy { boolean allows(String user, String resource); }
class OwnerPolicy implements PolicyStrategy {
    public boolean allows(String user, String resource) {
        return resource.startsWith(user + ":");
    }
}
class AccessController {
    private PolicyStrategy strategy;
    boolean check(String user, String resource) {
        return strategy.allows(user, resource);
    }
}
```

6.2 Python – Strategy

Listing 2: strategy.py (key excerpt)

```
class PolicyStrategy(ABC):
    @abstractmethod
    def allows(self, user: str, resource: str) -> bool: ...

class OwnerPolicy(PolicyStrategy):
    def allows(self, user, resource):
        return resource.startswith(f"{user}:")
```

6.3 JavaScript – Observer

Listing 3: observer.js (key excerpt)

```
class WorkflowEngine {
  run(job) {
    ["STARTED","PROCESSING","DONE"].forEach(s => this._notify(job, s));
  }
  _notify(job, status) {
    this._observers.forEach(o => o.onStatusChange(job, status));
  }
}
```

6.4 C++ – Observer

Listing 4: observer.cpp (key excerpt)

```cpp
struct WorkflowObserver {
    virtual void onStatusChange(const std::string& job, const std::string& status) = 0;
    virtual ~WorkflowObserver() = default;
};
class WorkflowEngine {
    std::vector<std::shared_ptr<WorkflowObserver>> observers;
public:
    void run(const std::string& job) {
        for (auto& s : {"STARTED","PROCESSING","DONE"}) notify(job, s);
    }
};
```

7 Three Evolution Scenarios

7.1 EV01 – Add Caching to Strategy

Change: Introduce a `CachedPolicyStrategy` decorator that memoises `allows()` results for 60 seconds.
Pattern effect: Decorator – newly introduced; wraps Strategy without modifying existing policy classes.
Applicable to: Java (decoration via interface), Python (`functools.lru_cache`), JavaScript (closure cache), C++ (template wrapper).

7.2 EV02 – Add Async Observer Notifications

Change: Observers notify asynchronously (non-blocking).
Pattern effect: Observer – extended; adds asynchronous dispatch.
Applicable to: Java (`CompletableFuture`), Python (`asyncio`), JavaScript (`EventEmitter`), C++ (`std::async`).

7.3 EV03 – Composite Policy (Multiple Strategies)

Change: Chain multiple policy strategies with AND/OR logic.
Pattern effect: Composite – newly introduced; allows composing any number of `PolicyStrategy` objects.
Applicable to: All four languages using same Composite interface.

8 Before/After Analysis

Evolution	Before	After	Delta LOC
EV01 Caching	No cache	CachedPolicy decorator	+12–18
EV02 Async	Sync notify	Async dispatch	+8–20
EV03 Composite	Single policy	CompositePolicy	+15–22

9 Refactoring Analysis

Smell: Long `check()` methods that also log; violated Single Responsibility.
Technique: Extract Method – logging extracted to the Observer layer.
Evidence: Strategy code handles only the allow/deny decision; Logger observer handles all output.
Behaviour-preservation: All four implementations produce identical console output before and after refactoring.

10 Four-Language Implementations

Lang	Interface mechanism	Strategy advantage	Observer advantage
Java	`interface` keyword, strict types	Compile-time enforcement	Easy list iteration
Python	ABC / Protocol (duck-typing)	Minimal boilerplate	Flexible attachment
JavaScript	Implicit duck-typing	No boilerplate, concise	`forEach` is idiomatic
C++	Pure virtual base class	Zero-overhead abstraction	`shared_ptr` lifetime safety

11 LLM Prompts and Outputs

11.1 Prompt Used

"Implement the Strategy and Observer patterns for an access-control and workflow notification system in [Language]. Keep it minimal – around 30–50 LOC. Use realistic class/method names from a data management domain."

11.2 Human Verification

Each implementation was manually compiled/run and output verified:

```
Java:       javac Strategy.java Observer.java && java Strategy && java Observer
Python:     python3 strategy.py && python3 observer.py
JavaScript: node strategy.js && node observer.js
C++:        g++ -std=c++17 strategy.cpp -o s && ./s && g++ -std=c++17 observer.cpp -o o && ./o
```

Output matches expected results for both allowed and denied access checks and for all three workflow status events.

12 Metrics

Language	Pattern	LOC	Classes	Coupling	Abstraction
Java	Strategy	45	3	0.12	High
Java	Observer	38	3	0.10	High
Python	Strategy	30	3	0.09	Medium
Python	Observer	28	3	0.08	Medium
JavaScript	Strategy	32	2	0.10	Low
JavaScript	Observer	30	2	0.09	Low
C++	Strategy	48	3	0.15	High
C++	Observer	42	3	0.13	High

12.1 Analysis

- **LOC:** C++ is most verbose (boilerplate, headers). Python is most concise.
- **Abstraction:** Java/C++ enforce interface contracts at compile time. Python/JS use duck-typing.
- **Coupling:** Python has the lowest coupling due to dynamic dispatch. C++ highest due to template dependencies.
- **Complexity:** Strategy: cyclomatic complexity 1–2 in all languages. Observer: 1–3.

13 Excel + CSV Dataset

Raw metrics are stored in `Part3/metrics.csv`. An Excel file (`Part3/metrics.xlsx`) can be generated from the CSV using any standard spreadsheet application.

Summary

The Strategy and Observer patterns express the same intent cleanly in all four languages. Python and JavaScript achieve the most concise implementations; Java and C++ provide stronger static guarantees at the cost of higher LOC. The patterns transfer naturally across language boundaries with only syntactic adaptations.